electronics

MDPI

Article

Formal Software Architecture Rule Learning: A Comparative Investigation between Large Language Models and Inductive Techniques

Christian Schindler * and Andreas Rausch

Institute for Software and Systems Engineering, Clausthal University of Technology, 38678 Clausthal-Zellerfeld, Germany
* Correspondence: christian.schindler@tu-clausthal.de

Abstract: This paper explores the application of inferring software architecture rules from examples using Machine Learning (ML). We investigate different methods from Inductive Rule Learning and utilize Large Language Models (LLMs). Traditional manual rule specification approaches are time-consuming and error-prone, motivating the need for automated rule discovery. Leveraging a dataset of software architecture instances and a meta-model capturing implementation facts, we used inductive learning algorithms and LLMs to extract meaningful rules. The induced rules are evaluated against a predefined hypothesis and their generalizability across different system subsets is investigated. The research highlights the capabilities and limitations of ML-based rule learning in the area of software architecture, aiming to inspire further innovation in data-driven rule discovery for more intelligent software architecture practices.

Keywords: software architecture; Inductive Rule Learning; constraint learning; first-order logic; Large Language Models

check for updates

Citation: Schindler, C.; Rausch, A. Formal Software Architecture Rule Learning: A Comparative Investigation between Large Language Models and Inductive Techniques. *Electronics* **2024**, *13*, 816. https://doi.org/10.3390/electronics13050816

Academic Editor: Juan M. Corchado

Received: 21 December 2023
Revised: 11 February 2024
Accepted: 18 February 2024
Published: 20 February 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Beyond the inductive learning methods discussed in the original paper [1], this journal extension addresses Large Language Models (LLMs) as another area for learning formal architectural rules. The ability to effectively define and enforce software architecture rules is crucial for the development of robust and maintainable software systems. Traditional approaches to rule definition often rely on manual specification, which can be time-consuming, error-prone, and limited in their ability to capture complex architectural constraints. In recent years, there has been growing interest in leveraging Machine Learning (ML) techniques to automatically learn and detect software architecture patterns [2] from data. A notable advantage of learning software architecture rules over manual rule specifications is the elimination of the need to start from scratch. We believe that evaluating and utilizing learned rules is more straightforward than formulating rules from the ground up. Moreover, an example-based learning approach facilitates the introduction of new concepts that may not be explicitly known, for instance, through literature or expert knowledge. These concepts become explicitly understood through the learning process, which is guided by a carefully chosen set of examples.

This paper explores the application of (i) Inductive Rule Learning (IRL) and (ii) LLMs, both subfields of ML, to infer simple software architecture rules from a Knowledge Base (KB) retrieved from a systems implementation, with a ground-truth annotation of architectural rule instances.

The goal of this paper is to investigate the feasibility and effectiveness of using ML techniques to extract meaningful and actionable architectural rules from a real-world software system. By leveraging the abundance of available architectural data, we aim to

empower software developers and architects with an automated approach to rule discovery that complements traditional manual rule definition.

Our research methodology involves collecting a dataset of software architecture instances, introducing a meta-model building the foundation of collected facts from the implementation, and the preparation of facts for the examples and counterexamples based on the provided software architecture documentation.

We employ state-of-the-art IRL tools and LLMs to perform rule induction from the dataset. In addition, we formulate a hypothesis about the architectural rule we expect to learn.

The tool-induced rules are assessed for alignment with the proposed hypothesis.

In addition, we investigate the ability to infer architecture rules in different subsets of the system to indicate the possibility of applying induced rules in different parts of the same system.

The contributions of this paper lie in the exploration of inductive learning techniques and LLMs abilities specifically tailored for software architecture rule inference. By demonstrating the potential of these techniques, we hope to stimulate further research and innovation in the field of data-driven rule discovery and pave the way for more intelligent and adaptive software architecture practices.

This paper aims to address the following Research Questions (RQs):

RQ1: How effective is the application of ML techniques in inferring software architecture rules from real-world systems?

RQ2: Can the induced rules be applied in different subsets of the same system, indicating their generalizability across architectural contexts?

Summary of results: We tested different IRL tools to determine whether they could learn a predefined hypothesis. Popper, one tool, demonstrated the ability to learn a similar hypothesis. In a subsequent trial, we exposed Popper to various subsets of software architecture examples. This revealed that the chosen examples influenced the learned rule. In 21 of 26 tasks, the original hypothesis was learned, highlighting its generalizability. Deviations from the original hypothesis stemmed from subsets with simpler explanations for differentiating positive and negative cases.

LLMs faced challenges in learning the anticipated software architecture rule in line with the hypothesis. An exception was Google Bard, which successfully learned a rule closely resembling the hypothesis when being exposed solely to positive examples. Contrary to Popper's approach, negative examples appeared to complicate the learning process for LLMs rather than offering further clarity on the task. This was evident as even Google Bard failed to formulate a rule that aligned closely with the initially stated hypothesis. In the preliminary experiment, we found that LLMs can induce a simple rule when the predicates in the examples have semantic meaning. However, if the predicates are obfuscated, the LLMs are unable to solve the task.

The outcomes of this research highlight the competencies and boundaries of IRL and LLMs in relation to software architecture. We not only present the findings but also delve into the dimensions that require increased complexity to make a substantial impact on data-driven software architecture rule inference.

The structure of this paper is organized as follows. Section 2 provides motivation and an overview of an integrated approach combining development practices with data-driven learning of architectural rules. Section 3 explores the foundations and related work in IRL, LLMs, and software architecture. Section 4 describes the experiments. Section 5 presents experiment results. Section 6 discusses the implications of the findings and outlines dimensions for increasing complexity in realistic scenarios. Finally, in Section 7, conclusions are drawn, contributions are summarized, and potential directions for future research are outlined.

2. Motivation

Software architecture is crucial in the development and maintenance of software systems, guiding their organization and implementation. However, defining and enforcing effective architectural rules becomes challenging as systems grow in size and complexity.

Traditional rule definition approaches rely on manual specification, which is time-consuming, error-prone, and limited in capturing complex constraints [3]. To address this, automated techniques are needed to discover and enforce architectural rules.

ML and data-driven techniques, in particular, show promise. They generalize from observed examples to detect recurring patterns, dependencies, and constraints.

This research aims to explore the feasibility and effectiveness of applying ML algorithms to infer software architecture rules. Our goal is to advance data-driven software architecture practices and enable more intelligent and adaptive approaches to software system design and evolution.

Figure 1 illustrates the integration of an architectural rule learning mechanism to enhance the verification of architectural consistency within development teams. On the right-hand side, we have a developer who is responsible for building and maintaining the system. In addition to writing code, the developer annotates the implemented architectural concepts, providing a foundation for performing consistency checks between the implementation and the designated software architecture for the corresponding part of the system [4,5]. This combination of manual annotation and automated rule learning ensures that architectural guidelines are adhered to, promoting a more robust and maintainable software architecture.

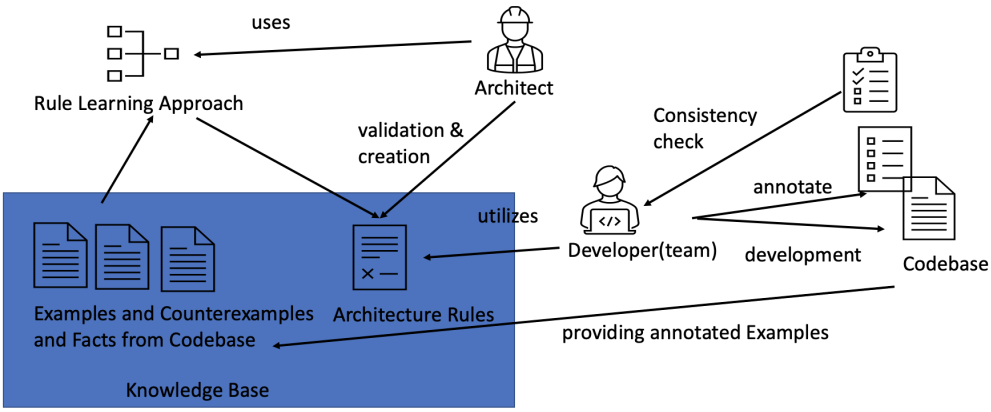

Figure 1. Data-driven learning approach in software architecture.

The process involves annotating architectural examples, which are then fed into a KB. Leveraging these examples, a data-driven learning approach [6] utilizing ML can be employed to learn rules. Architects are able to interact with the KB, configure learning tasks and tools, and validate the learned rules. The generated rules can be seamlessly integrated back into the development cycle, allowing for their practical application and enforcement within the software development process. By incorporating the learned rules, architects can ensure adherence to architectural guidelines and promote consistency and quality throughout the software system.

3. Fundamentals and Related Work

In this section, we review related work across three areas: IRL, LLMs, and software architecture. This overview sets the context for our study and illustrates the interplay between these key topics.

3.1. Inductive Rule Learning

IRL is a process that aims to learn logical rules to generalize training examples, with the objective of maximizing positive examples and minimizing counterexamples [7]. IRL

systems employ different strategies such as bottom–up, top–down, or a combination thereof to explore the hypothesis space and derive generalized rules from the training data. Learned solutions bias towards learning simple (short) rules [8]. The task of learning horn clauses is shown to be $\sum_2^P$ complete [9]. A more recent approach in IRL is meta-level IRL, where the learning problem is formulated as a meta-level logic problem that involves reasoning about programs [7,10,11] or incorporating neural networks into inductive learning [12,13], directly allowing learning tasks from raw data without the need for a pre-existing KBs.

Another recent direction involves scaling inductive learning on large KBs [14] to predict the missing links within them. This area has seen significant contributions in terms of the scalability of learning algorithms, which aim to learn hypotheses that do not necessarily apply to the entire provided KB. Instead, they focus on predicting missing links to enhance the information present in the database.

3.2. Large Language Models

LLMs are advanced types of language models known for their excellent performance in understanding and processing human language, as Fan [15] noted in 2023. These models are particularly good at making sense of information with only a few examples, especially when working with text and data tables, a capability Chen [16] pointed out in 2022. Interestingly, the larger these models become, the better they perform [17]. LLMs are also used in translating languages, where they analyze patterns in language data to produce translations, a method Brants [18] discussed in 2007. **Generative Pre-trained Transformers (GPTs)** are strong models for processing natural language. They are based on a special structure known as the transformer architecture and have many potential uses (e.g., in education, healthcare, and the entertainment industry) and challenges (e.g., high computational requirements in pre-training and inference, lack of interpretability, and ethical considerations) [19]. There has been a lot of research on GPT, especially in computer science. It is closely linked to fields like artificial intelligence, language processing, and deep learning, as Cano [20] pointed out in 2023. GPT is not just for language tasks; it is also being used in engineering design. It can help come up with new design ideas, something Gill [21] found in 2023. Zhu [22] in 2021 also noted that GPT is helpful in the early stages of coming up with design concepts.

3.3. Software Architecture

Architectural design is a critical step in software development, with a significant impact on the resulting system [23]. It involves the creation of a high-level abstraction of system topology, functionality, and behavior, which serves as the foundation for subsequent detailed design and implementation [24]. Architectural design should predominantly be guided by established and proven design patterns and best practices [25], as well as other factors like requirements [26] and stakeholder concerns [27]. Trade-offs often need to be involved to leverage conflicting goals. A well-considered design is pivotal during the creation and evolution of a system, especially when navigating changes and adaptations.

Software architecture documentation plays a foundational role by explicitly documenting and effectively communicating the architecture. It serves as a vital aid for decision making throughout the software system's development and maintenance. Both informal and formal [28–34] approaches, contribute to this documentation process.

In **reverse engineering software architecture** [35,36], the objective is to glean insights into design choices, employed patterns, and inherent constraints. This endeavor not only facilitates a deeper understanding of the software system at hand but also supports its ongoing maintenance. The insights garnered from reverse engineering empower informed architectural decisions.

To our knowledge, no approach has specifically been proposed for **learning architectural rules** through examples. Existing approaches focus on architectural reconstruction, classification, and abstract representation learning from code, as well as mining software architectural knowledge from sources like Stack Overflow. For example, there are approaches

utilizing data mining and ML techniques [2,37] and others that extract knowledge from alternative sources [38]. However, there is a gap in the literature regarding directly learning architectural rules from examples.

4. Experiment

In this section, we present our hypothesis and experiments leveraging various tools to learn from real-world data in TEAMMATES [39]. We assess the effectiveness (RQ1) and transferability (RQ2) of the learned rules across different architectural contexts.

4.1. Experimental Setup

We aim to demonstrate the learning of software architecture rules using an inductive approach with a community-sourced dataset. The software system TEAMMATES provides relevant documentation [40] on a software architecture rule called isAllowedToUse [41]. It specifies that particular modules can use another modules within the system. The documentation also includes the explicit counterexamples of the rule, identifying modules that are not permitted to use certain others. The architecture documentation provides the mappings of modules to packages.

To conduct our experiment, we extracted examples from the documentation and transformed them into logic fact in the format of isAllowedToUse(usingPackage, usedPackage) and not(isAllowedToUse(usingPackage, usedPackage)).

The meta-model shown in Figure 2 is the foundation for the facts we collect from any given implementation. This describes the known domain concepts (*Class* and *Package*) among the relations between the concepts. We utilize the meta-model to derive unary and binary predicates. Unary predicates represent the concepts *Class* and *Package*, whereas binary predicates denote the relationships between these concepts. For instance, *importsClass* connects the concept *Class* to itself, indicating that *Classes* can import other *Classes*. Additionally, the cardinalities specify the nature of these relationships. In this scenario, a given *Class* can be imported by *0..** (i.e., zero or more) *Classes*. This means that, while it is not mandatory for any other *Class* to import a given *Class*, multiple *Classes* can indeed import it. Similarly, the cardinality at the other end of the same connection is also *0..**, implying that a class can import multiple *Classes*.

Figure 2. Domain model of the types and their relationships

The meta-model is intentionally small, focusing on the concepts and relations we need to define a hypothesis that covers the architecture rule *isAllowedToUse(A, B)*. In Section 6.2, we will address the different dimensions of how the complexity of the given approach can be increased so that the approach can scale to learning rules for more complex architectural concepts.

To illustrate the relationship between the meta-model and the KB, we provide a small example using a code snippet (Listing 1) and the resulting facts (Listing 2). This example showcases how the meta-model establishes the foundation for the predicates used in the KB, as well as how concrete facts from a Java implementation are represented.

Listing 1 presents a Java code snippet, focusing on the class *Gatekeeper* located within the *teammates.logic.api* package. Notably, the implementation details of the class are excluded from the fact base as they are not relevant in this context. Additionally, the snippet highlights the import statement for the *teammates.common.util.Const* class, providing further insights into the class dependencies and relationships.

Listing 2 displays the resulting facts obtained from the code snippet. In total, four facts are extracted from the code. Firstly, we observe the fact representing the package *teammates.logic.api*. Secondly, there is a fact indicating that *teammates_logic_api_GateKeeper* is a class. The third fact signifies that the *GateKeeper* class imports the *Const* class from the *teammates_common_utils* package. The last fact states that *GateKeeper* is located within the *teammates_logic_api* package.

Listing 1. Code snippet of the Java class GateKeeper.

```
package teammates.logic.api;
import teammates.common.util.Const;

public class GateKeeper {
        //out of scope
}
```

Listing 2. Resulting facts in the knowledge base.

```
1 package(teammates_logic_api).
2 class(teammates_logic_api_GateKeeper).
3 importsClass(teammates_logic_api_GateKeeper,
      teammates_common_util_Const).
4 containsClass(teammate_logic_api,
      teammates_logic_api_GateKeeper).
```

The KB comprises 8340 facts, including 41 *package* facts, 40 *containsPackage* facts, 767 *class* facts, 767 *containsClass* facts, and 6725 *importsClass* facts.

Our **working hypothesis** is that the permitted usage of packages can be inferred by examining the import mechanism, as Java classes need to import other classes in order to utilize them. Equation (1) presents the hypothesis based on the meta-model (see Figure 2). A hypothesis consists of two parts: the rule head and the rule body.

The rule head, in our case *isAllowedToUse(a,b)*, is a single predicate, contrasting with the rule body that often consists of multiple predicates connected through logical operators.

The rule body states that both *a* (the using package) and *b* (the used package) must be of the type *package*. The rules introduce two free variables (*c* and *d*) that need to be of the type *class*. Additionally, the rule body imposes constraints on the containment of variables c and d within packages a and b (using the predicates *containsClass(a, c)* and *containsClass(b, d)*), and requires the *importsClass(c,d)* relationship between the free variables to hold. In such a hypothesis, the rule head is implied by the rule body, so if the rule body holds, the rule head also holds.

$$\begin{aligned} \forall a \forall b : isAllowedToUse(a,b) \Leftarrow package(a) \\ \wedge package(b) \wedge \exists c : class(c) \\ \wedge \exists d : class(d) \wedge containsClass(a,c) \\ \wedge containsClass(b,d) \wedge importsClass(c,d) \end{aligned} \tag{1}$$

We employed several **tools from the research community** to learn rules for the *isAllowedToUse* architecture rule. Tools from the IRL community we use including Aleph [42] version 6, AMIE3 [43,44], AnyBURL [14] version 23-1, ILASP [10] version 4, and Popper [11] version 2.1.0. While Aleph, ILASP, and Popper were compatible with our existing KB of facts (refer to Listings 1 and 2), and adjustments were necessary for AnyBURL and AMIE3 as they are designed to learn rules on a knowledge graph.

To adapt the facts, we transformed them into a knowledge graph format comprising triples that consist of a subject, predicate, and object. For predicates with a parity of two in our KB, we performed the following transformation: *containsClass(package, class)* resulted in the triple *package containsClass class).* On the other hand, unary predicates were handled differently by introducing the predicate *isA* and using the original predicate as the object of the triple. For instance, *package(P)* became the triple *P isA Package*.

We also utilize LLMs for the task of learning software architecture rules. The tested LLMs are Google Bard [45] version 2023.10.23, OpenAI ChatGPT 3.5 Turbo (GPT-3.5) as well as ChatGPT 4 (GPT-4) [46]. All experiments involving LLMs were conducted using the official websites that OpenAI and Google provide. The experiments did not set specific parameters such as temperature or frequency_penalty.

4.2. Conducted Experiments

Preliminary experiment with LLMs: In our initial investigation, we explored the capability of LLMs to derive first-order logical rules from given examples (and counterexamples) alongside a KB of facts. Our chosen dataset for this first study was the *kinship-pi* dataset [11], focusing on grandparent relationships among individuals. This dataset utilizes the foundational predicates *father/2* and *mother/2*, which signify the parental relationship, for instance, *father(bob, carl)* indicates that Bob is Carl's father.

Our experiments were designed to evaluate the proficiency of LLMs in learning first-order logical rules from a combination of examples and provided facts. We adopted varied experimental setups. Initially, we supplied the LLMs with the complete dataset, observing their capability to assimilate the task. Subsequently, we employed an alternative interaction method, the *Flipped Interaction Pattern* [47] wherein we do not provide all facts upfront. Instead, the task and examples were presented, allowing the LLMs to request specific facts, which were then supplied by a human operator.

Furthermore, we repeated these experiments with an obfuscated version of the *kinship-pi* dataset. Here, predicate names and individual names were replaced with alphabet letters (e.g., *mother* became *a*, *father* became *b*, and so forth), effectively stripping the data of their semantic clarity.

Our preliminary findings revealed that GPT-3.5 successfully learned the rule from the original dataset but struggled with the obfuscated data. Furthermore, Google Bard had more difficulty in deriving a rule for the grandparent relationship across all preliminary experiments examples, and similarly to GPT-3.5, did not perform well with the obfuscated data.

These results suggest that LLMs may rely on semantic information, which is not explicitly provided in the task or examples, to effectively resolve the given task. This reliance seems more pronounced when dealing with data that retain their semantic integrity, as evidenced by the better performance with meaningful predicate names. Still, the general capability to induce first-order logic rules in this experiment motivated the investigation of the architectural rule isAllowedToUse(A, B) and to incorporate LLMs into the conducted experiments.

Experiment 1: In the first experiment, we limited the examples used to learn a hypothesis to come exclusively from the *logic* package specified in the architecture. The examples we selected were constrained to directly specify the *isAllowedToUse* rule between packages. However, the ground data also included examples that specified the rule between packages that did not directly contain classes, but rather contained packages with classes inside them (e.g., *isAllowedToUse(teammates_ui, teammates_common)*). The **objective** of this experiment is to learn a simple architectural rule, denoted by Equation (1), utilizing different IRL tools and LLMs available and to answer RQ1.

Figure 3 presents an overview of Experiment 1. In its lower-right corner, we provide a key to the notation used. A box with a wavy bottom line symbolizes a document, such as our developed meta-model, software architecture documentation, among others. Activities like

the definition of a hypothesis, learning tasks, the creation of the KB, etc., are represented by boxes with rounded corners. These activities and documents are interconnected by arrows.

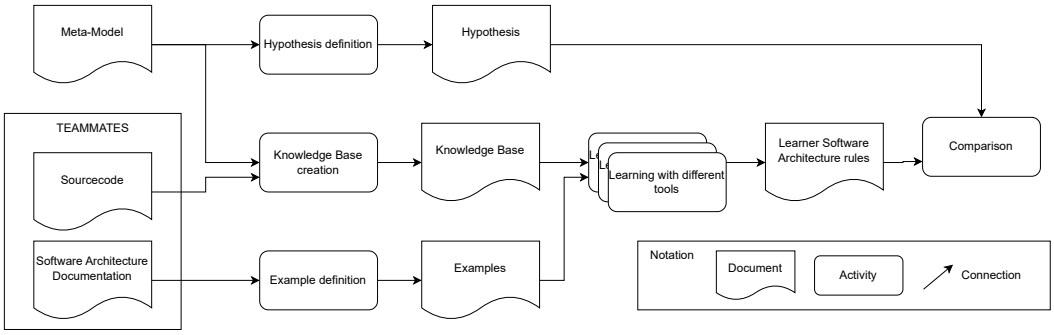

Figure 3. Visualization of Experiment 1.

In the upper section of the figure, the utilization of the meta-model in defining the hypothesis is depicted. This hypothesis represents the software architecture rule anticipated to be learned by the tools. It plays a crucial role in the comparison activity, where the rules learned by the tools are assessed against it.

From the software system TEAMMATES, we extract both the source code and the existing software architecture documentation. The source code, alongside the meta-model, is employed to extract logical facts that form a model consistent with the meta-model. The software architecture documentation is used to delineate both the positive and negative instances of the software architecture rule. These instances, along with the KB, are integral to the learning phase. During this phase, we format the information to meet the specific requirements of each tool being tested, in accordance with their learning specifications. Similarly to the hypothesis formulation process, the rules learned by a tool are subsequently fed into the comparison step, enabling a thorough evaluation against the original hypothesis.

We intentionally confined this first experiment to examples dealing with packages that directly contain classes. All the examples used in this experiment were packages located within the teammates.logic package. In this setup, we tested multiple tools. Depending on the capabilities of each tool, we learned rules using only positive examples of the isAllowedToUse rule, and in the cases where the tools allowed counterexamples, we also learned rules in a second run by including the same amount of counterexamples to create a balanced dataset. All tools learned on the same examples (Listing 3).

Listing 3. Examples and counterexamples used in Experiment 1.

```
#positive examples
isAllowedToUse(teammates_logic_api, teammates_logic_core).
isAllowedToUse(teammates_logic_backdoor, teammates_logic_api).

#negative examples
isAllowedToUse(teammates_logic_core, teammates_logic_backdoor).
isAllowedToUse(teammates_logic_api, teammates_logic_backdoor).
```

The experimental setups for IRL tools and LLMs differ, and we elaborate on the specifics of the conducted experiments in the following section. Details regarding IRL tool configurations and interactions with the LLMs can be accessed in the accompanying reproduction package (https://github.com/schindlerc/ML-Based-Software-Architecture-Rule-Learning (accessed on 18 December 2023)).

For the IRL tools, we adhered to the provided documentation to configure them and subsequently allowed them to operate, inducing rules based on the given parameters.

In the context of utilizing LLMs for rule learning, our approach commenced with a preliminary feasibility test using a simplified dataset. This initial step served as a foundation before advancing to the more complex task of learning software architecture rules.

OpenAI offers a web-based tokenizer tool (https://platform.openai.com/tokenizer, accessed on 17 February 2024) that facilitates an understanding of how a given text can be tokenized. This tool provides information on the number of tokens and utilizes color to visually separate individual tokens within the given string. For our latest fact in Listing 2 ($containsClass(teammate_logic_api, teammates_logic_api_GateKeeper)$), the tool indicates a token count of 15. How the fact is tokenized can be seen in Figure 4.

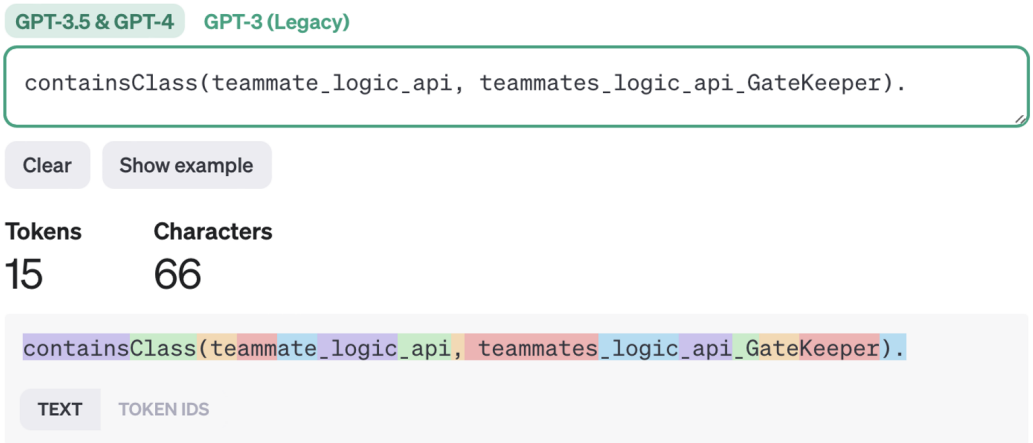

Figure 4. OpenAI tokenizer tokenizing a fact (**upper** part) into the tokens (**lower** part).

Considering the scale of the KB containing 8340 facts, the total token count expands to approximately 125,100 tokens. This surpasses the token limit of GPT-3.5, capped at 16,385 tokens, and nearly reaches the maximum context window size of GPT-4 (version 1106) with a limit of 128,000 tokens. It is important to note that GPT-4 with that context window size is still in its preview stage. In [48], it is shown that current LLMs does not robustly make use of information in long input contexts, focusing on the information provided in the beginning or the end of the input context, while according lower priority to the information located in the middle of a long input contexts provided.

To overcome the challenges of the limited token window size of the models, we use prompt engineering patterns, like the flipped interaction pattern [47]. This pattern helps the LLMs by guiding them to ask questions and gather the needed information for a task. In this flipped interaction approach, instead of providing the model with all the available information, we task the LLMs with asking questions to collect needed information to fulfill the task of inducing a first-order logical rule.

In Listing 4, we show the prompt that started our conversations with the LLMs. The prompt sets the task for LLMs to ask for facts until they determine a first-order logical rule. After that, the LLMs list all available details and give examples, both positive and negative, showcasing their ability to ask questions and obtain task-related information on their own.

In the experiment, we will not allow the LLMs to ask questions forever (even if stated in the prompt) but will terminate the conversations if (i) the LLM will repeatedly ask for the same facts; (ii) it repeatedly states rules that are not correct and does not improve even when told that it is wrong.

By conducting this experiment, we sought to evaluate the capabilities of different tools and determine whether they can accurately learn the desired rule using positive examples only (Table 1) or by incorporating both positive and negative examples (Table 2) to create a balanced dataset. Learning duration and accuracy of the learned rules are given in Table 3.

Listing 4. Flipped interaction prompt.

```
I would like you to ask me for specific facts, so you are able
to induce a first order logical rule that explains all
the positive and none of the negative examples.
You should ask questions until this condition is met
or to achieve this goal (alternatively, forever)
Allowed predicates to ask for are:
package/1
containsPackage/2
class/1
containsClass/2
importsClass/2

Ask me questions one at a time.
The positive examples are:
isAllowedToUse(teammates_logic_api, teammates_logic_core).
isAllowedToUse(teammates_logic_backdoor, teammates_logic_api).

The negative examples are:
isAllowedToUse(teammates_logic_core, teammates_logic_backdoor).
isAllowedToUse(teammates_logic_api, teammates_logic_backdoor).
```

Experiment 2: In our second experiment, we expanded the scope of the learning task by considering additional modules. In addition to the logic package, we included the storage package, resulting in a total of six packages being considered. Consequently, the number of examples increased from 6 to 30 as we examined pairwise observations regarding the permissibility of one package by another.

The **objective** of this experiment was to investigate the possibility of learning the same hypothesis from different subsets of examples. We aimed to determine whether a rule learned in one part of the system could also hold true for instances of the architectural rule in a different part of the system. By exploring this aspect, we sought to assess the transferability and generalizability of the learned rules across various parts within a software system to address RQ2.

Figure 5 provides a detailed overview of Experiment 2. Mirroring the methodology of Experiment 1, we constructed the KB and examples using the source code and software architecture documentation, aligned with the meta-model. However, a key distinction in Experiment 2 is the preliminary step of selecting examples, which was performed multiple times to generate various sets of examples. Each set served as input for a separate learning run using the IRL tool, Popper. During the comparison phase of this experiment, we analyzed the rules learned from the different subsets. This included a qualitative examination of the examples from which the rules were derived, offering insights into the learning outcomes.

To conduct the experiment, we randomly generated the subsets of six examples of the architectural rules and utilized Popper to learn the rules based on the given examples. To ensure balanced subsets, each randomly generated subset contained at least two positive and two negative examples.

In total, we created 26 subsets for learning rules. Each subset was used to learn a hypothesis, and the resulting hypotheses, along with the number of subsets that yielded them, are presented in Table 4.

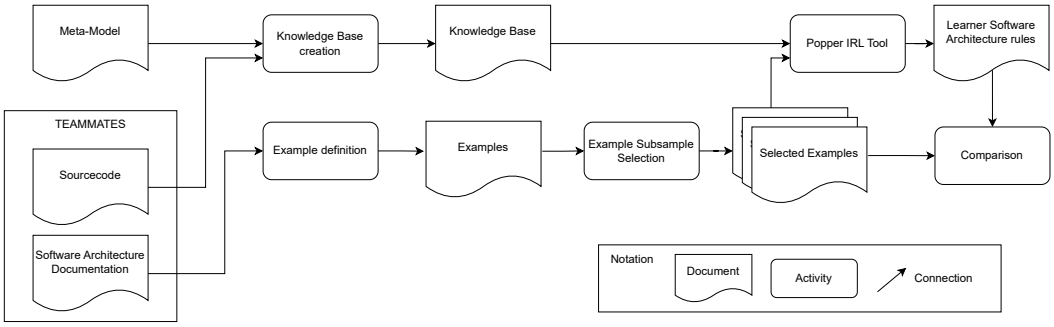

Figure 5. Visualization of Experiment 2.

5. Results

In Experiment 1, the rules learned are compiled into two categories and presented in Table 1 for those exclusively derived from positive examples, and in Table 2 for those formulated from both examples and counterexamples.

Table 1. Learned rule bodies for the rule head *isAllowedToUse(A, B)* in Experiment 1, exclusively derived from positive examples for each tool.

Tool	Learned Rule Body
Aleph	$package(B).$
AMIE3 1. Rule AMIE3 2. Rule	$containsPackage(G, A) \wedge containsPackage(G, B).$ $isA(A, H) \wedge isA(B, H).$
AnyBURL	$isA(A, X) \wedge isA(Y, X) \wedge containsPackage(Y, B)$
ILASP (V4)	$package(B) \vee containsPackage(C, A).$
Popper	$package(B) \wedge package(A).$
Google Bard	$package(A) \wedge containsClass(A, D) \wedge containsClass(B, C)$ $\wedge importsClass(D, C).$
GPT-3.5	$containsPackage(A, C) \wedge containsClass(C, B)$
GPT-4	$containsPackage(Z, A) \wedge containsPackage(Z, B)$

Table 2. Learned rule bodies for the rule head *isAllowedToUse(A, B)* in Experiment 1, derived from positive examples and counterexamples for each tool.

Tool	Learned Rule Body
Aleph 1. Rule Aleph 2. Rule	isAllowedToUse(teammates_ logic_ api,teammates_ logic_ core). isAllowedToUse(teammates_ logic_ backdoor,teammates_ logic_ api).
AMIE3	No run with explicit counter examples
AnyBURL	No run with explicit counter examples
ILASP (V4)	UNSATISFIABLE
Popper	$importsClass(D, C) \wedge containsClass(A, D) \wedge containsClass(B, C).$
Google Bard	$package(A) \wedge containsPackage(A, C) \wedge class(B)$ $\wedge containsClass(C, B) \wedge not(A = B).$
GPT-3.5	$containsClass(A, B) \wedge package(A) = teammates_logic_core.$
GPT-4	$package(A) \wedge package(B) \wedge containsClass(A, X) \wedge containsClass(B, Y)$ $\wedge not(importsClass(A, B)) \wedge not(importsClass(B, A)).$

Furthermore, it is important to highlight that learning with counterexamples was only possible with three of the IRL tools. Among these three, only two tools successfully generated a rule. Specifically, Popper learned a rule with variables, while Aleph returned a

hypothesis consisting of the two positive examples provided but did not learn a rule with variables in the second setting. This variation in performance suggests that the capabilities of the IRL tools differed in their ability to handle the inclusion of counterexamples.

The results of Experiment 2 are presented in Table 4, summarizing learned architecture rules for the different subsets of examples.

5.1. Results of Experiment 1

Aleph includes a feature called induce_modes, which enables the induction of modes for both head and body predicates. In our study, we utilized this feature to extract the suggested modes from Aleph for the given data. Upon applying Aleph to the dataset, it learned a simple rule for *isAllowedToUse(A, B)* where the implication was that *B* represents a package (*package(B)*). This rule aligns with the meta-model's restriction (refer to Figure 2), as the *isAllowedToUse* relation connects packages. Note that Aleph achieved accurate results solely based on positive examples.

While Aleph has the capability to learn from both positive and negative examples, the rules it learned in this scenario are not usable. The outcome generated by Aleph for positive and negative examples solely provides the positive examples, rather than learning a rule that implies *isAllowedToUse(A, B)* with variables A, B, and potential unbound variables. As a result, the rules learned by Aleph with negative examples did not meet the desired criteria for applicability.

AMIE3 learns rules based on a knowledge graph without the explicit provision of negative examples, resulting in rules learned solely from positive examples. It is important to note that the format of the examples provided differs for AMIE3, as well as for AnyBurl, which we will discuss later.

AMIE3 learned two rules. The first rule implies *isAllowedToUse(A, B)* based on the existence of *G*, which contains both *A* and *B*. Given the restrictions on the selected examples, where all are packages within *teammates.logic*, this rule accurately reflects a valid assignment for *G*.

The second rule learned by AMIE3 has a unique characteristic that arises from translating our meta-model into triples, as required by AMIE3. This rule states that *isAllowedToUse(A, B)* is implied by both *isA(A, H)* and *isA(B, H)*. In the knowledge graph, there are triples such as *teammates_logic_core isA package*, which leads to the conclusion that the only suitable assignment for *H* is *package*. Thus, the learned rule reflects that *A* and *B* are of the same type, namely, packages.

However, these rules do not fully align with the essence of our stated hypothesis, as they do not consider the *importsClass(X, Y)* predicate and its connection to the head predicate, nor the involvement of the bound variables *A* and *B*.

Similarly to AMIE3, **AnyBURL** also operates on a knowledge graph composed of triples. Consequently, the remark regarding the altered input format, where triples like *teammates_logic_core isA package* are used instead of *package(teammates_logic_core)*, is also applicable here. AnyBURL, unlike the other tested tools, does not require or provide a specific configuration for the head predicate of interest. Instead, it explores all possible relations within the learning phase, aiming to derive as many rules as possible. Out of the 2532 rules learned by AnyBURL, only one rule corresponds to the target head predicate *isAllowedToUse*, which is presented in Table 1. This rule suggests that *isAllowedToUse(A, B)* is implied by *A* and *Y* is both an *X* (as indicated by *isA*) and *Y* containing a package (*containsPackage B*). However, similarly to the previously discussed tools, this learned rule does not capture the essence of our stated hypothesis. It is important to note that AnyBURL does not support explicit learning with counterexamples, thereby precluding a second run to refine the learned rules.

ILASP has the capability to learn rules from both examples and counterexamples. However, in our experiments, ILASP was only able to learn rules based on the two positive examples provided. When presented with the task of learning from positive and negative examples, ILASP responded with the learning task being *unsatisfiable*. Therefore, the rule

learned is solely based on positive examples, as shown in Table 1. The learned rule from ILASP suggests that *isAllowedToUse(A, B)* is either implied by a disjunction *B* being a package (*package(B)*) or by *A* being a package contained within *C* (*containsPackage(C, A)*). However, neither the first nor the second alternative in the rule body captures the essence of the hypothesis we formulated. Thus, the rules derived from ILASP do not align with our intended idea.

Popper stands out among the tested IRL tools as the only one capable of learning rules that incorporate variables in the learning task, including counterexamples, for the architecture rule *isAllowedToUse(A, B)*.

The rule learned based on positive examples alone differs from the rule learned using both examples and counterexamples. The rule derived from positive examples only implies *isAllowedToUse(A, B)* when both *A* and *B* are packages. Although this rule is conditionally induced by the meta-model, it does not capture the essence of our hypothesis and is not considered suitable for abstracting the provided architectural rule examples.

The rule learned using both positive and negative examples proves to be the most promising outcome of the entire experiment. This rule includes the predicate *importsClass(D, C)* with the free variables *D* and *C* in the rule body. The incorporation of this predicate and the presence of free variables enhance the rule's potential to align with our intended hypothesis.

In addition, the rule learned by Popper connects the variables *C* and *D* to be classes contained in *A* and *B* (*containsClass(A, D)* and *containsClass(B, C)*, respectively). This matches the last three predicates of the hypothesis in Equation (1), except for the interchange of the free variables *C* and *D* to *d* and *c*, which does not alter the way that the predicates are connected in our hypothesis and the rule learned by Popper. Furthermore, it is important to note that by the interchange of *C* and *D* with *d* and *c*, we are referring to the order of the variables and not the alternating use of upper and lower cases. This change in variable names does not affect the way that the predicates are connected in both our hypothesis and the rule learned by Popper.

Compared to our hypothesis, the learned rule does not contain the first part stating that the bound variables *A* and *B* are packages, and it does not explicitly state that *C* and *D* need to exist (using the existential quantifier $\exists$). However, the rule learned by Popper still captures the essence of our hypothesis by connecting the variables *C* and *D* as classes contained in *A* and *B*, respectively, as well as incorporating the *importsClass(D, C)* predicate with the free variables *D* and *C* in the rule body.

Existence is achieved by finding suitable assignments for the respective variables, and these unary predicates, which set the types of the variables (whether they represent classes or packages), are predefined by the simple meta-model and the binary predicates they are used in. Consequently, it is implicit that these aspects of our hypothesis hold in every example (and counterexample). Therefore, they do not contribute to distinguishing the provided examples from the provided counterexamples.

GPT-3.5 demonstrated its capacity to acquire rules through two modes of the experiment: (i) from positive examples and (ii) from additional counterexamples. However, when exclusively learning from positive examples, it generated a rule that is in conflict with our meta-model. Specifically, the induced rule erroneously posits that *C* contains a *class B*, i.e., *containsClass(C, B)*, implying an *isAllowedToUse* relationship between a *package* and a *class*. Contrary to this, our meta-model unambiguously stipulates that the *isAllowedToUse* dependency exclusively operates between packages.

Furthermore, the rule learned from both provided examples and counterexamples exhibits an anomalous component. It asserts that *package(A)* equals *teammates_logic_core*, a syntactical oddity. This departure from conventional syntax is noteworthy, as it introduces a value, *teammates_logic_core*, into a predicate intended to yield a truth value of true or false, raising questions about the interpretability of such assignments.

While GPT-3.5 effectively processed the experimental data and formulated rules suggesting the presence of *isAllowedToUse(A, B)*, it was unable in both instances to induce rules that comprehensively encompassed the positive examples provided.

Unlike its predecessor, **GPT-4** is capable of learning rules through the presentation of examples alone or through the inclusion of counterexamples at the outset of its training process. In both scenarios, the rules acquired during training demonstrate the capability to encompass the given positive examples, yielding flawless accuracy when positive examples alone are provided. However, when counterexamples are introduced into the training data, GPT-4 struggles to formulate a rule that effectively excludes these counterexamples, resulting in a rule set with an accuracy of 50 percent.

The comparison of our initial hypothesis with the observed rules yields intriguing insights. Firstly, the rule formed solely from positive examples has nothing in common with our hypothesis. The learned rule only expects a shared entity, *Z*, encompassing both *A* and *B* (*containsPackage(Z, A)* and *containsPackage(Z, B)*), but lacks further commonalities. In contrast, the rules derived from both examples and counterexamples exhibit greater alignment with our original hypothesis. Both not only recognize *A* and *B* as packages but also introduce *X* and *Y* as the respective *containsClass* for *A* and *B*. However, a critical deviation emerges in the remainder of the learned rule. It erroneously asserts that neither *importsClass(A, B)* nor *importsClass(B, A)* occur, which contradicts our understanding. This is a fundamental misinterpretation, as *importsClass* is intended to function with *class* types, not *package* types. In addition, the learned rule does not contain the part of the hypothesis that defines the *importsClass* relationship between the *classes X* and *Y*, i.e., *importsClass(X, Y)*.

In our first experiment, **Google Bard** learned rules for both setups and obtained different results. When given only positive examples, Google Bard figured out a rule that is quite similar to what we expected. It learned that *A* should have a *class D*, i.e., *containsClass(A, D)*, *B* should have a *class C*, i.e., *containsClass(B, C)*, and *D* needs to be imported into *C*, i.e., *importsClass(D, C)*. Google Bard's rule also added that *A* should be a package, which aligns with our hypothesis and meta-model (see Figure 2, Equation (1)).

However, the rule that Google Bard learned with counterexamples was not as good as the first one. It wrongly included the idea that *B* should be a *class*, i.e., *class(B)*, which does not match our meta-model. According to our meta-model, the architectural relationship we are looking at (*isAllowedToUse(A, B)*) only exists between *packages*. This mistake makes the rule wrong for all the positive examples we have.

Table 3 provides the **learning times and rule accuracy**.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \tag{2}$$

Table 3. Learning duration and rule accuracy for each tool in Experiment 1.

Tool	**With Examples**		**With Examples and Counterexamples**	
	Learning Time (s)	**Accuracy**	**Learning Time (s)**	**Accuracy**
Aleph	0.010	1.0	0.097	-
AMIE3	0.150	1.0	-	-
AnyBURL	<1.000	1.0	-	-
ILASP	4.922	1.0	1918.680	-
Popper	<1.000	1.0	<1.000	1.0
	# User Messages	Accuracy	# User Messages	Accuracy
Google Bard	6 messages	1.0	44 messages	0.0
GPT-3.5	19 messages	0.0	54 messages	0.0
GPT-4	22 messages	1.0	44 messages	0.5

Except for ILASP, all IRL tools had learning times below one second in Experiment 1. Aleph was the fastest with 0.01 s, while ILASP took 4.922 s. In the second run, Aleph and Popper had low learning times, but ILASP took 1918 s. This long learning time can be

attributed to the exploration of the entire hypothesis space. This results in its inability to learn a rule and declaring it as unsatisfiable. All learned rules learned by IRL tools—except for Aleph where no parameterized rules were learned—achieved 100% accuracy.

For the tested LLMs, the outcomes exhibit greater variability. GPT-3.5 struggled to consistently formulate precise rules. Google Bard, in contrast, succeeded in developing an accurate rule but only for positive examples. GPT-4 demonstrated a mixed performance, achieving perfect accuracy (1.0) when working with positive examples, but its accuracy dropped to 0.5 with the inclusion of counterexamples, due to the misclassification of all counterexamples.

Assessing the learning time in interactions with chatbot-like interfaces for LLMs presents a challenge, as it is not directly comparable to the learning time of IRL tools. Therefore, we have opted to report the number of messages that the user sent to the chatbot. The total number of messages (# User Messages in the table) exchanged in these chats is approximately double this figure, considering the back-and-forth dialogue between the user and the LLM during the rule-learning process. The duration of each LLM experiment, which includes retrieving relevant responses from the LLM, composing replies, and documenting the chat history for the reproduction dataset, consistently ranged from 10 to 15 min.

After conducting experiments with tools from the distinct ML domains of IRL and LLMs, it is enlightening to highlight some fundamental differences in the learning process's dynamics and user experience.

With IRL tools, the learning process predominantly involves setting up the test data (KB and examples) for the task at hand, enabling the tools to autonomously navigate through the hypothesis space until a rule is identified. Once initiated, this task proceeds without requiring any further input from the user or expert.

In contrast, LLMs, particularly when employing the flipped interaction mode in our experiments, initiate the learning process with an initial prompt, followed by an iterative exchange of messages between the LLM and the user. This interactive approach not only accommodates a diverse range of messages from both parties but also supports adaptive responses during the rule-learning phase. We recognize significant potential in further exploring these interactive possibilities, as they have a profound impact on both the learning journey and its outcomes.

Regarding RQ1, our findings indicate that Popper, utilizing examples and counterexamples, learned a rule closely aligned with the proposed hypothesis. However, other IRL tools learned rules that did not fully capture the essence of the architectural rule, yet still encompassed the presented examples. Notably, these tools generated rules almost instantly for the small-scale experiments, achieving perfect accuracy. In the context of learning software architectural rules, LLMs exhibited significant limitations. Despite being extensively trained on substantial datasets, LLMs frequently encounter challenges in accurately learning software architectural rules, closely aligning with our hypothesis. Notably, only in one experimental setup (Google Bard, absent counterexamples) did the essence of our hypothesis become evident in the learned rule. We observed ML techniques from IRL and LLMs that showed promise in learning effective rules in our experiments. However, many tested approaches tended towards simpler explanations for the examples provided. This inclination resulted in the derivation of basic rules that ultimately fell short of meeting our expectations.

5.2. Results of Experiment 2

For the second experiment, we solely focused on using Popper, as it was the only tool from Experiment 1 that learned a rule closely aligned with our hypothesis on examples and counterexamples. The results of Experiment 2, presented in Table 4, showcase the learned rule and the number of subsets that led to its discovery. To summarize the learned rules, we applied the commutative law by reordering the body predicates and aligned the free

variables. Interestingly, we observed that 21 out of 26 subsets produced the same rules as Popper learned in the first experiment, considering both examples and counterexamples.

Table 4. Experiment 2: Popper's learned rule bodies for the rule head *isAllowedToUse(A, B)* and their amount for different example subsets.

Learned Rule Body	# of Subsets
$containsClass(A, C) \wedge containsClass(B, D) \wedge importsClass(C, D).$	21
$containsPackage(C, B) \wedge containsPackage(C, A).$	2
$importsClass(D, C) \wedge package(A) \wedge$ $containsClass(B, C) \wedge importsClass(C, D).$	2
$containsClass(B, C) \wedge importsClass(D, E) \wedge containsClass(A, D) \wedge$ $importsClass(D, C) \wedge importsClass(C, E).$	2

For the remaining five cases where different rules were learned, we examined the examples within each subset to determine the reasons behind the variation. In two instances, Popper derived the rule *isAllowedToUse(A, B)* from a free variable *C* that contained both packages *A* and *B*. These instances corresponded to accurate rules, as all the examples within each subset belonged to the same package (either *teammates.logic* or *teammates.storage*). Notably, this was not the case for the counterexamples, which contributed to the differentiation in learned rules.

In the case of the other two rules that have been learned (twice and once, respectively), we noticed a common counterexample *not(isAllowedToBeUsed(teammates_logic_core, teammates_storage_entity)* that appeared in all three subsets. However, we believe that this particular negative example may be inaccurately labeled or could represent an inconsistency within the documented architecture and the implementation.

We identified a class (*FeedBackResponseLogic*) within the package *teammates.logic.core* that imports and interacts with objects from (*FeedbackResponse*), which is located in the package *teammates.storage.entity*. This violates the architectural rule, as the expected behavior suggests no allowance to use objects from the other package. However, in the implementation, we observed the usage of classes from both packages, which contradicts the rule of not being *allowedToUse*. Due to this inconsistency, the expected rule fails to distinguish between the provided examples and the counterexamples. This violation raises doubts about the accuracy of the labeling and highlights a potential discrepancy between the documented architecture and its actual implementation.

As a consequence, the rules learned in this particular case tend to be more complex. This is primarily because the examples and the inconsistent counterexample exhibit greater similarity to each other in terms of the facts that can be represented using the simple meta-model (Figure 2). Popper needs to learn a more intricate rule to differentiate between the examples and counterexamples within the specific subsets.

In response to RQ2, our findings demonstrate that Popper is capable of inducing rules from various subsets of examples within the same system, suggesting a degree of generalizability across different architectural contexts. However, it is important to note that the specific subset employed can influence the learned rules and, consequently, their generalizability. For instance, if the IRL tool can find a simpler explanation, such as "all positive examples have packages located in the same package", it prioritizes this shorter explanation over more nuanced rules.

6. Validity Considerations and Future Directions

While our experiments have indeed produced promising results, it is essential to recognize and address potential factors that could impact their validity.

6.1. Threats to Validity

Limited Dataset Scope: The effectiveness demonstrated by our model on the small, single-source dataset might not entirely encapsulate its potential when confronted with larger and more diverse datasets. The inherent constraints of the dataset's scale and diversity could influence the model's performance on real-world, multifaceted data distributions.

Single Architectural Rule: Our investigation solely focused on a solitary, straightforward software architectural rule. As a consequence, the applicability of our findings to the learnability of considerably more intricate rules, characterized by greater complexity and a wider spectrum of concrete implementations, may be limited based on the scope of the conducted experiment.

With respect to LLMs, solely one interaction tactic (the Flipped Interaction Pattern) was used in the experiments, as the token scope is a limiting factor on how to provide software architecture-related tasks on learning software architecture rules.

By acknowledging and addressing these aspects, we ensure a comprehensive understanding of the boundaries and implications of our study's outcomes. This discernment is vital for translating our findings into broader contexts and for guiding future research in the field.

In the following section, we outline ways to enhance the complexity of our approach and tackle the stated threads.

6.2. Dimensions of Complexity Enhancement

To elevate the demonstrated approach of learning software architecture rules to a realistic level, there are several dimensions in which complexity needs to be increased.

One aspect is increasing the size of the meta-model. The current meta-model introduced in the experiment has limitations in terms of the types and relations it represents. It lacks the ability to encompass the fundamental concepts of object-oriented programming, such as inheritance, attributes, methods, and their visibility. Consequently, a significant amount of information from the implementation is currently not captured in the KB. To address this, a valuable extension of the meta-model is required to incorporate necessary information, enabling the expression of more complex architectural rules. By expanding the meta-model to include these crucial aspects of object-oriented programming, we can enhance the richness and depth of the KB. This, in turn, will allow for the learning of more sophisticated architectural rules that better align with the intricacies of real-world software systems.

Another dimension in increasing the complexity of learning architectural rules involves the ability to utilize quantifiers in the learned rules. The inclusion of quantifiers enables the learning of more precise rules by allowing the expression of conditions that must hold for every occurrence and we gain the capability to capture and enforce more nuanced and specific patterns within the software architecture. Ultimately, the use of quantifiers enhances the precision and accuracy of the learned rules, enabling a deeper understanding of the architectural properties of the system.

Addressing the need to handle more heterogeneous examples represents a significant dimension of increasing complexity. This requirement arises from various factors. One contributing factor is the expansion of the meta-model and the growing complexity of the collected facts derived from the implementation. As the KB expands in size and intricacy, we can expect the examples to exhibit a greater diversity compared to the relatively constrained flexibility observed in the conducted experiments.

Another reason for the necessity of handling more heterogeneous examples stems from the nature of complex architectural rules. While we have demonstrated the ability to learn simple software architecture rules using only a few examples and counterexamples, this may not hold true for more intricate rules. Therefore, addressing the challenge of handling more heterogeneous examples is crucial for advancing the learning of software architecture rules. This task becomes particularly pertinent considering the expanding meta-model and the inherent complexities associated with real-world architectural scenarios. By tackling this challenge, we can enhance the robustness and applicability of learned architectural rules in diverse software development contexts.

7. Conclusions and Future Work

In conclusion, this research has shed light on the potential of inductive learning techniques for inferring software architecture rules from known examples. Our findings highlight the capabilities of IRL and LLMs in capturing and detecting a simple architectural rule. Through experiments and analyses, we have shown that these techniques can learn meaningful rules that align with the expected architectural hypotheses, albeit with some limitations and variations in learned rules solely based on (positive) examples or on specific subsets of examples. The results are promising, and in combination with the efforts of IRL approaches to learning in large KBs and the fast pace in LLMs, intriguing questions about the scalability of the proposed approach arise. Moreover, we identified dimensions for increasing the complexity of rule inference, including expanding the meta-model to encompass more architectural concepts, handling heterogeneous examples, and incorporating quantifiers for more precise rule expressions. Future work aimed at addressing the complexity dimensions discussed could involve conducting a more extensive test to explore the scalability of the approach. Additionally, this would provide an opportunity to further investigate and mitigate current threats to validity. These research outcomes provide valuable insights in the field of data-driven software architecture.

By further exploring and advancing learning approaches, we can unlock the potential of (semi-)automated rule discovery, leading to more robust, maintainable, and adaptive software systems. Looking ahead, future research can focus on addressing the identified dimensions, refining the rule learning process, and exploring novel techniques to enhance the expressiveness, scalability, and generalizability of learned architectural rules. We recognize the promising potential of hybrid methodologies combining LLMs with IRL strategies. Specifically, we aim to explore various interaction modes with LLMs, such as advanced prompting techniques. This will include more active interactions (from the user) and providing additional comprehensive information in the prompts such as the typesystem of the meta-model or the whole universe of entities of a given task. Another area for enhancement involves integrating validation checks into the LLM learning process. This will ensure that any learned rules are subject to syntactic verification, and rules that violate the constraints of the typesystem or deviate from the principles of constructing first-order logical rules will be rejected. By continuing to push the boundaries of data-driven rule inference, we can empower software developers and architects with powerful tools for effective software architecture governance and evolution.

Author Contributions: Conceptualization, data curation, formal analysis, investigation, software, methodology, validation, investigation, writing—original draft preparation, writing—review and editing, visualization, and project administration C.S.; funding acquisition and supervision, A.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data (knowledge base), logs, and instructions on how to use the the tools are released in the GitHub-repository https://github.com/schindlerc/ML-Based-Software-Architecture-Rule-Learning (accessed on 18 December 2023).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

GPT	Generative Pre-trained Transformer
IRL	Inductive Rule Learning
KB	Knowledge Base
LLM	Large Language Model
ML	Machine Learning
R	Research Question

References

1. Schindler, C.; Rausch, A. Towards Inductive Learning of Formal Software Architecture Rules. In Proceedings of the 57th Annual Hawaii International Conference on System Sciences (HICSS), Hawaiian Village, HI, USA, 3–6 January 2024; pp. 7302–7311.
2. Komolov, S.; Dlamini, G.; Megha, S.; Mazzara, M. Towards Predicting Architectural Design Patterns: A Machine Learning Approach. *Computers* **2022**, *11*, 151. [CrossRef]
3. Tibermacine, C.; Sadou, S.; Dony, C.; Fabresse, L. Component-based Specification of Software Architecture Constraints. In Proceedings of the 14th International ACM Sigsoft Symposium on Component Based Software Engineering, Boulder, CO, USA, 21–23 June 2011; pp. 31–40.
4. Deiters, C.; Dohrmann, P.; Herold, S.; Rausch, A. Rule-based Architectural Compliance Checks for Enterprise Architecture Management. In Proceedings of the 2009 IEEE International Enterprise Distributed Object Computing Conference, Auckland, New Zealand, 1–4 September 2009; pp. 183–192.
5. Pruijt, L.; Wiersema, W.; van der Werf, J.M.E.; Brinkkemper, S. Rule Type Based Reasoning on Architecture Violations: A Case Study. In Proceedings of the 2016 Qualitative Reasoning about Software Architectures (QRASA), Venice, Italy, 5–8 April 2016; pp. 1–10.
6. Schindler, M.; Schindler, C.; Rausch, A. A Formalism for Explaining Concepts through Examples based on a Source Code Abstraction. *Int. J. Adv. Softw.* **2023**, *16*, 1–22.
7. Cropper, A.; Dumančić, S. Inductive Logic Programming At 30: A New Introduction. *J. Artif. Intell. Res.* **2022**, *74*, 765–850. [CrossRef]
8. Quinlan, J.R.; Cameron-Jones, R.M. FOIL: A Midterm Report. In Proceedings of the Machine Learning: ECML-93: European Conference on Machine Learning, Vienna, Austria, 5–7 April 1993; Proceedings 6; Springer: Berlin/Heidelberg, Germany, 1993; pp. 1–20.
9. Gottlob, G.; Leone, N.; Scarcello, F. On the Complexity of Some Inductive Logic Programming Problems. In Proceedings of the Inductive Logic Programming: 7th International Workshop, ILP-97, Prague, Czech Republic, 17–20 September 1997; Proceedings 7; Springer: Berlin/Heidelberg, Germany, 1997; pp. 17–32.
10. Law, M.; Russo, A.; Broda, K. The ILASP System for Inductive Learning of Answer Set Programs. *arXiv* **2020**, arXiv:2005.00904.
11. Cropper, A.; Morel, R. Learning programs by learning from failures. *Mach. Learn.* **2021**, *110*, 801–856. [CrossRef]
12. Cunnington, D.; Law, M.; Lobo, J.; Russo, A. Inductive Learning of Complex Knowledge from Raw Data. *arXiv* **2022**, arXiv:2205.12735.
13. Dai, W.Z.; Muggleton, S. Abductive Knowledge Induction from Raw Data. In Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21, Virtual, 19–26 August 2021; Zhou, Z.H., Ed.; International Joint Conferences on Artificial Intelligence Organization; pp. 1845–1851.
14. Meilicke, C.; Chekol, M.W.; Ruffinelli, D.; Stuckenschmidt, H. Anytime Bottom-Up Rule Learning for Knowledge Graph Completion. In Proceedings of the IJCAI, Macao, China, 10–16 August 2019; pp. 3137–3143.
15. Fan, L.; Li, L.; Ma, Z.; Lee, S.; Yu, H.; Hemphill, L. A Bibliometric Review of Large Language Models Research from 2017 to 2023. *arXiv* **2023**, arXiv:2304.02020.
16. Chen, W. Large Language Models are few(1)-shot Table Reasoners. *arXiv* **2022**, arXiv:2210.06710.
17. Zhao, W.X.; Zhou, K.; Li, J.; Tang, T.; Wang, X.; Hou, Y.; Min, Y.; Zhang, B.; Zhang, J.; Dong, Z.; et al. A Survey of Large Language Models. *arXiv* **2023**, arXiv:2303.18223.
18. Brants, T.; Popat, A.; Xu, P.; Och, F.J.; Dean, J. Large Language Models in Machine Translation. In Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), Prague, Czech Republic, 28–30 June 2007; pp. 858–867.
19. Yenduri, G.; Ramalingam, M.; ChemmalarSelvi, G.; Supriya, Y.; Srivastava, G.; Maddikunta, P.K.R.; DeeptiRaj, G.; Jhaveri, R.H.; Prabadevi, B.; Wang, W.; et al. Generative Pre-trained Transformer: A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges, and Future Directions. *arXiv* **2023**, arXiv:2305.10435.
20. Cano, C.A.G.; Castillo, V.S.; Gallego, T.A.C. Unveiling the Thematic Landscape of Generative Pre-trained Transformer (GPT) Through Bibliometric Analysis. *Metaverse Basic Appl. Res.* **2023**, *2*, 33. [CrossRef]
21. Gill, A.S. Chat Generative Pretrained Transformer: Extinction of the Designer or Rise of an Augmented Designer. In Proceedings of the International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Boston, MA, USA, 20 August 2023.
22. Zhu, Q.; Luo, J. Generative Pre-Trained Transformer for Design Concept Generation: An Exploration. *Proc. Des. Soc.* **2021**, *2*, 1825–1834. [CrossRef]
23. Bosch, J.; Molin, P. Software architecture design: Evaluation and transformation. In Proceedings of the ECBS'99. IEEE Conference and Workshop on Engineering of Computer-Based Systems, Nashville, TN, USA, 7–12 March 1999; pp. 4–10.
24. Fu, Y.; Dong, Z.; He, X. A Method for Realizing Software Architecture Design. In Proceedings of the 2006 Sixth International Conference on Quality Software (QSIC'06), Beijing, China, 27–28 October 2006; pp. 57–64.
25. Cervantes, H.; Kazman, R. *Designing Software Architectures: A Practical Approach*; Addison-Wesley Professional: Indianapolis, IN, USA, 2016.
26. Liao, L. *From Requirements to Architecture: The State of the Art in Software Architecture Design*; Department of Computer Science and Engineering, University of Washington: Washington, DC, USA, 2002; pp. 1–13.

27. Van Vliet, H.; Tang, A. Decision making in software architecture. *J. Syst. Softw.* **2016**, *117*, 638–644. [CrossRef]
28. Feiler, P.H.; Lewis, B.; Vestal, S.; Colbert, E. An Overview of the SAE Architecture Analysis & Design Language (AADL) Standard: A Basis for Model-Based Architecture-Driven Embedded Systems Engineering. In Proceedings of the Architecture Description Languages: IFIP TC-2 Workshop on Architecture Description Languages (WADL), World Computer Congress, Toulouse, France, 22–27 August 2004; Springer: Berlin/Heidelberg, Germany, 2005; pp. 3–15.
29. Feiler, P.H.; Gluch, D.P. *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language*; Addison-Wesley: Cambridge, UK, 2012.
30. Kherbouche, M.; Molnár, B. Formal Model Checking and Transformations of Models Represented in UML with Alloy. In Proceedings of the Modelling to Program: Second International Workshop, M2P 2020, Lappeenranta, Finland, 10–12 March 2020; Revised Selected Papers 1; Springer: Berlin/Heidelberg, Germany, 2021; pp. 127–136.
31. Gamble, R.F.; Stiger, P.; Plant, R. Rule-based systems formalized within a software architectural style. *Knowl.-Based Syst.* **1999**, *12*, 13–26. [CrossRef]
32. Bowen, J.P. Z: A formal specification notation. In *Software Specification Methods: An Overview Using a Case Study*; Springer: London, UK, 2001; pp. 3–19.
33. Jackson, D. Alloy: A lightweight object modelling notation. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **2002**, *11*, 256–290. [CrossRef]
34. Heyman, T.; Scandariato, R.; Joosen, W. Security in Context: Analysis and Refinement of Software Architectures. In Proceedings of the 2010 IEEE 34th Annual Computer Software and Applications Conference, Seoul, Republic of Korea, 19–23 July 2010; pp. 161–170.
35. Stringfellow, C.; Amory, C.; Potnuri, D.; Andrews, A.; Georg, M. Comparison of Software Architecture Reverse Engineering Methods. *Inf. Softw. Technol.* **2006**, *48*, 484–497. [CrossRef]
36. Singh, R. A Review of Reverse Engineering Theories and Tools. *Int. J. Eng. Sci. Invent.* **2013**, *2*, 35–38.
37. Maffort, C.; Valente, M.T.; Bigonha, M.; Hora, A.; Anquetil, N.; Menezes, J. Mining Architectural Patterns Using Association Rules. In Proceedings of the International Conference on Software Engineering and Knowledge Engineering, Boston, MA, USA, 27–29 June 2013.
38. Ali, M.; Mushtaq, H.; Rasheed, M.B.; Baqir, A.; Alquthami, T. Mining software architecture knowledge: Classifying stack overflow posts using machine learning. *Concurr. Comput. Pract. Exp.* **2021**, *33*, e6277. [CrossRef]
39. TEAMMATES—Online Peer Feedback/Evaluation System for Student Team Projects. Available online: https://teammatesv4.appspot.com/web/front/home (accessed on 1 February 2024).
40. Herold, S. SAEroConRepo. 2017. Available online: https://github.com/sebastianherold/SAEroConRepo (accessed on 14 May 2023).
41. Schröder, S.; Buchgeher, G. Discovering Architectural Rules in Practice. In Proceedings of the 13th European Conference on Software Architecture—Volume 2, New York, NY, USA, 9 September 2019; ECSA'19, pp. 10–13.
42. Srinivasan, A. The Aleph Manual. 2001. Available online: https://www.dcc.fc.up.pt/~ines/aulas/1920/TAIA/The%20Aleph%20Manual%20V6.pdf (accessed on 19 February 2024).
43. Galárraga, L.A.; Teflioudi, C.; Hose, K.; Suchanek, F. AMIE: Association Rule Mining under Incomplete Evidence in Ontological Knowledge Bases. In Proceedings of the 22nd International Conference on World Wide Web, Rio de Janeiro, Brazil, 13–17 May 2013; pp. 413–422.
44. Lajus, J.; Galárraga, L.; Suchanek, F. Fast and Exact Rule Mining with AMIE 3. In Proceedings of the The Semantic Web: 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, 31 May–4 June 2020; Proceedings 17; Springer: Berlin/Heidelberg, Germany, 2020; pp. 36–52.
45. Manyika, J.; Hsiao, S. An overview of Bard: An Early Experiment with Generative AI. Available online: https://ai.google/static/documents/google-about-bard.pdf (accessed on 14 December 2023).
46. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. GPT-4 Technical Report. *arXiv* **2023**, arXiv:2303.08774.
47. White, J.; Fu, Q.; Hays, S.; Sandborn, M.; Olea, C.; Gilbert, H.; Elnashar, A.; Spencer-Smith, J.; Schmidt, D.C. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *arXiv* **2023**, arXiv:2302.11382.
48. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the Middle: How Language Models Use Long Contexts. *arXiv* **2023**, arXiv:2307.03172.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.